ECE 411: Computer Organization and Design MP Out-of-Order Processor Implementation

Chirag Maheshwari
Computer Engineering, UIUC
chiragm4@illinois.edu

Akshat Mehrotra
Computer Engineering, UIUC
akshatm4@illinois.edu

Dash Patrick Kamriani Beard
Computer Engineering, UIUC
dashpk2@illinois.edu

***Abstract*—This report documents Team BP's implementation of an out-of-order RISC-V processor as part of ECE 411: Computer Organization & Design. We present our design decisions, implementation progress across multiple checkpoints, and the advanced features we incorporated. Our Explicit Register Rename (ERR) approach enabled several performance enhancements, including pipelined cache integration, early branch recovery, a superscalar back-end, and a partially implemented split load-store queue. We analyze performance impacts of each feature and discuss challenges encountered during implementation. This report provides insights into our design process, performance analysis, and lessons learned throughout this comprehensive processor design project.**

***Index Terms*—Out-of-Order Execution, Explicit Register Rename, RISC-V, RV32IM, Early Branch Recovery, Superscalar Architecture, Computer Architecture**

I. Introduction

Modern processor designs employ sophisticated techniques to extract instruction-level parallelism (ILP) and improve performance beyond what in-order execution can achieve. Out-of-order (OoO) execution is a cornerstone technology that allows processors to execute instructions as soon as their operands are available, rather than strictly following program order. This approach significantly improves CPU performance by hiding latency and maximizing resource utilization.

This report documents our team's journey in designing and implementing a synthesizable RISC-V 32-bit out-of-order processor with M-extension support (multiplication and division) over an 8-week period. We adopted the Explicit Register Rename (ERR) approach, which uses a physical register file and explicit register mappings to track register values and dependencies. Our implementation progressed through multiple checkpoints, from initial block diagrams and fetch-stage implementation to a complete functional processor with advanced features such as early branch recovery and a superscalar back-end.

Throughout this project, we applied fundamental computer architecture concepts while gaining practical experience in RTL design, verification, and optimization. The following sections detail our design approach, implementation challenges, performance analysis, and technical insights gained from this comprehensive processor design project.

II. Project Overview

Our team opted for an Explicit Register Rename (ERR) approach for our out-of-order processor design based on several considerations. The ERR technique proved more intuitive for our team members and had stronger support among course staff, potentially easing the debugging process through office hours. This approach uses a physical register file (PRF) with explicit mapping tables to track register values and dependencies.

A. Design Philosophy

We established a structured development workflow to maximize our productivity as a team. Our process involved:

- Initial planning sessions to define module interfaces and responsibilities
- Independent development of sub-modules by individual team members
- Regular integration sessions to combine modules and verify interactions
- Comprehensive verification using both directed and randomized test approaches

This modular development approach allowed us to parallelize work effectively while maintaining a cohesive overall design. When faced with integration challenges, we would dynamically reorganize, with some team members focusing on debugging while others worked ahead on upcoming features.

B. Project Timeline

The project was structured across multiple checkpoints, each with specific deliverables:

- Checkpoint 1: Block diagram design and initial RTL implementation (fetch stage, FIFO, burst adapter)
- Checkpoint 2: Support for arithmetic and logical operations including M-extension instructions
- Checkpoint 3: Complete functional processor with memory operations and control flow
- Advanced Features: Performance optimizations and architectural enhancements

We successfully completed Checkpoints 1 and 2 on schedule, but encountered challenges during Checkpoint 3 that required additional debugging time. This delay impacted our advanced feature implementation timeline, though we still managed to develop several significant enhancements.

III. Design Description

A. Overview

Our out-of-order processor implements the RISC-V RV32IM instruction set and follows the Explicit Register Rename (ERR) methodology. The processor pipeline consists of the following key stages:

- Fetch: Retrieves instructions from memory into an instruction queue
- Decode: Identifies instruction types and extracts relevant fields
- Dispatch: Allocates physical registers, updates mapping tables, and issues to reservation stations
- Execute: Performs operations in specialized execution units (ALU, multiplier, divider, branch, memory)
- Complete: Updates results via Common Data Bus (CDB) and resolves dependencies
- Commit: Retires instructions in order and handles architectural state updates

The ERR approach allows us to track and manage dependencies explicitly through register mapping tables. Our design includes a reorder buffer (ROB) that tracks in-flight instructions and ensures precise exception handling and in-order commit despite out-of-order execution.

B. Milestones

1) Checkpoint 1: For the first checkpoint, we designed a comprehensive block diagram of our proposed processor architecture and implemented the initial components of our design: the fetch stage, a burst-DRAM deserializer, and a parameterizable FIFO.

The fetch stage interfaced with memory through our burst adapter to retrieve instructions and place them into the instruction queue. The parameterizable FIFO was designed with configurable depth and width to accommodate various data types throughout the processor. We implemented full and empty signals with correct timing to ensure proper handshaking between pipeline stages.

Our implementation passed all the test cases for this checkpoint, and we received positive feedback on our thorough verification approach. In particular, our randomized testbench for the FIFO was highlighted as a strength, as it tested corner cases such as simultaneous push/pop operations and boundary conditions.

2) Checkpoint 2: For the second checkpoint, we expanded our implementation to support arithmetic and logical operations, including the RISC-V M-extension for multiplication and division. This required developing several critical components:

- Decode unit: Parsed instruction fields and identified operation types
- Dispatch logic: Managed register renaming using our free list and mapping tables
- Reservation stations: Tracked operand availability and instruction readiness
- Execution units: Implemented ALU, multi-cycle multiplier, and divider
- Common Data Bus (CDB): Facilitated result distribution to dependent instructions

The M-extension implementation required careful consideration of signed/unsigned operations and multi-cycle execution. We implemented a multi-cycle divider using a sequential algorithm and a multiplier that supported the required RISC-V multiply operations (MUL, MULH, MULHU, MULHSU).

To verify this checkpoint, we developed a Python script to generate random arithmetic and logical instruction sequences. We also enhanced our verification infrastructure by integrating an expanded random testbench from mp_verif that included support for M-extension instructions.

Our implementation for Checkpoint 2 successfully passed all test cases, demonstrating correct functionality for the supported instruction subset.

3) Checkpoint 3: Checkpoint 3 required us to complete the processor implementation by adding support for memory operations and control flow instructions. This involved developing:

- Memory Management Unit (MMU): Handled load/store operations to memory
- Load-Store Queue (LSQ): Managed memory operation ordering and dependencies
- Branch Unit: Executed control flow instructions and handled mispredictions
- Reorder Buffer (ROB): Tracked in-flight instructions and managed architectural state

Our initial implementation included a basic load-store queue that ensured memory operations executed in order and a branch unit with an always-not-taken prediction strategy. While functionally correct in our local testing environment, we encountered several issues when submitting to the autograder:

- Discrepancies between our local tests and the autograder tests revealed subtle bugs in our handling of the zero register (x0)
- We discovered redundant write-enable signals in our execution units that occasionally caused incorrect behavior
- Most critically, our primarily combinational dispatch logic created combinational loops that weren't detected in simulation but prevented synthesis

These issues prevented us from earning points for Checkpoint 3, though we were able to identify and address these problems in the following weeks.

4) Advanced Design Features: After resolving the issues from Checkpoint 3, we focused on implementing advanced features to improve the performance of our processor:

a) Pipelined Cache: Our initial implementation already included a pipelined cache, but we identified and fixed a performance bottleneck in our execution unit FSMs. Each execution unit would transition from IDLE to COMPLETE states regardless of whether it was selected by the CDB arbiter, creating unnecessary stalls. By modifying this behavior to eliminate redundant state transitions, we improved the efficiency of our memory operations, allowing single-cycle hits to be properly utilized.

b) Early Branch Recovery (EBR): One of our most significant advanced features was the implementation of Early Branch Recovery. Traditional branch recovery waits until a mispredicted branch reaches the head of the ROB before flushing, which wastes cycles on instructions that will ultimately be discarded. Our EBR implementation enhances recovery performance by:

- Assigning each branch instruction a unique identification bit
- Tagging subsequent instructions with a mask indicating which unresolved branches they follow
- Enabling immediate flush of younger instructions once a branch misprediction is detected

This feature significantly improved performance on branch-heavy benchmarks by reducing the recovery latency. While it increased power and area usage, the IPC improvements justified this tradeoff for most workloads.

c) Superscalar Back-End: To further improve performance, we designed a superscalar back-end capable of executing multiple instructions simultaneously. This required significant modifications to several components:

- Multiple ALU instances to execute arithmetic operations in parallel
- Redesigned reservation stations to track and issue multiple ready instructions
- Multiple Common Data Buses (CDBs) to distribute results simultaneously
- Modified ROB to support committing multiple instructions per cycle

The superscalar back-end successfully synthesized at 588 MHz, demonstrating the potential for high-frequency operation. However, we encountered some functional correctness issues with multiply/divide operations in certain edge cases that we couldn't fully resolve before the deadline.

d) Split Load-Store Queue (Partial Implementation): Our final advanced feature was a split Load-Store Queue, which separates loads and stores into distinct queues. This approach allows loads to execute out of order with respect to older stores when there are no address conflicts, potentially hiding memory latency.

The implementation included:

- Separate load and store queues with independent management
- Address conflict detection logic to maintain memory consistency
- Age-tracking mechanism similar to our EBR implementation to identify instruction ordering

Though we were unable to fully integrate this feature with our EBR and baseline CPU within the time constraints, individual tests showed promising IPC improvements for memory-intensive benchmarks.

e) Performance Analysis: We conducted extensive performance analysis to compare our various implementations against benchmarks. Figure 8 shows the performance metrics (IPC, Delay, and Power) for our CP3 baseline, the provided baseline, our EBR implementation, and our Multi-CDB implementation across three key benchmarks.

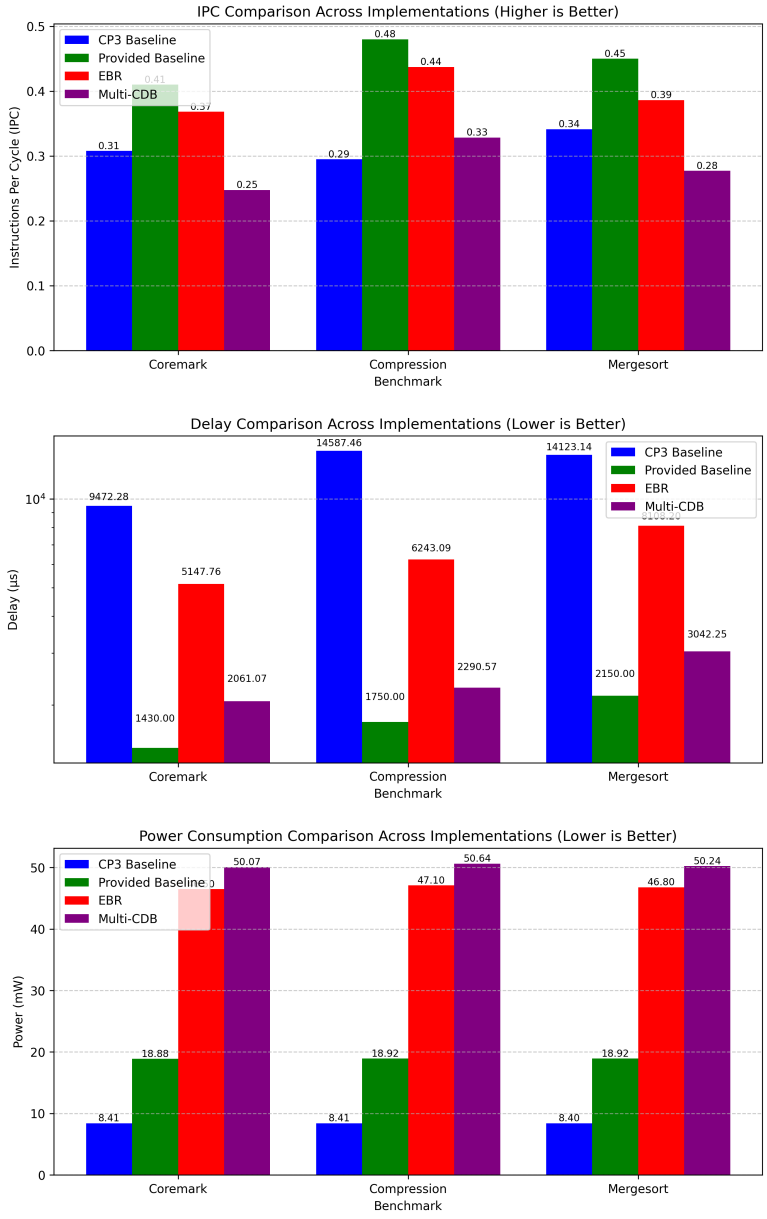

Fig. 1. Performance comparison of our different processor implementations. The charts show IPC (higher is better), Delay in microseconds (lower is better), and Power consumption in mW (lower is better) across three benchmarks.

As shown in Figure 1, our Early Branch Recovery implementation achieved significant improvements in both IPC and delay compared to our CP3 baseline. For the Coremark benchmark, EBR improved IPC by 19.6% and reduced delay by 45.7%. For the Compression benchmark, we observed a remarkable 48.3% improvement in IPC and a 57.2% reduction in delay. These improvements came at the cost of increased power consumption, which is a common trade-off in processor design.

Our Multi-CDB implementation showed mixed results.

While it achieved high clock frequency through improved critical path optimization, it suffered from functional issues that limited its IPC. However, the Multi-CDB design demonstrated impressive delay reduction across all benchmarks, particularly in Coremark where it reduced delay by 78.3% compared to our CP3 baseline.

The provided baseline generally showed better IPC than our implementations, highlighting areas for future optimization. However, our advanced features demonstrated competitive performance in specific metrics, validating our design approach.

IV. Additional Discussion / Observations

Despite our progress, we identified several areas for potential improvement that would have enhanced our processor's performance:

a) Performance Counters: Implementing detailed performance counters would have provided valuable insights into bottlenecks and guided our optimization efforts. Specifically, tracking metrics such as:

- Dispatch stalls due to full reservation stations or ROB
- Branch misprediction rates and recovery latency
- Cache hit/miss statistics
- CDB contention frequency

This profiling data would have allowed us to make more informed decisions about which advanced features to prioritize.

b) Branch Prediction: While our Early Branch Recovery mechanism improved branch misprediction handling, implementing a sophisticated branch predictor would have further reduced mispredictions. A relatively simple G-share predictor could have significantly improved performance on branch-heavy workloads without substantial implementation complexity.

c) Integration Challenges: One of our most significant challenges was the integration of independently developed features. While our modular approach facilitated parallel development, combining these modules revealed subtle interaction issues that were difficult to debug. In retrospect, more frequent integration cycles with comprehensive regression testing would have identified these issues earlier.

d) Critical Path Optimization: Our dispatch logic initially created a significant critical path that limited our maximum clock frequency. Through careful redesign and logic simplification, we were able to improve this, but further optimizations could have been made to balance pipeline stages more effectively.

V. Contributions

- **Chirag Maheshwari**: Block diagram design, fetch stage implementation, burst adapter, memory arbiter, M-extension execution units, CP3 baseline debugging, multiple CDB implementation, multi-commit ROB, and random instruction generation script
- **Dash Patrick Kamriani Beard**: Decode stage, memory management unit (MMU), and split LSQ implementation
- **Akshat Mehrotra**: Block diagram contributions, dispatch logic, reservation stations, branch unit with early branch recovery (EBR), and random testbench development

VI. Conclusion

Our implementation of an out-of-order RISC-V processor showcases both the challenges and opportunities in modern processor design. Through this project, we gained valuable insights into the complexities of managing instruction dependencies, optimizing critical paths, and balancing performance against power and area constraints.

The progression from a basic fetch unit to a complete out-of-order processor with advanced features like early branch recovery and superscalar execution represents a significant accomplishment. While we encountered challenges, particularly during integration and debug phases, these difficulties provided valuable learning experiences in verification methodology and system-level thinking.

Performance analysis of our advanced features demonstrated the impact of architectural decisions on processor efficiency. Early branch recovery provided substantial speedups for control-flow-intensive workloads, while our superscalar backend showed promise for exploiting instruction-level parallelism in compute-bound applications.

This project provided us with practical experience using industry-standard tools and methodologies for hardware design and verification. The skills developed through this implementation—from RTL design to debugging complex timing issues—have direct applications in professional hardware development environments.

We would like to thank the course staff for providing this enriching educational experience in computer architecture and hardware design. The opportunity to design a complete processor from first principles has deepened our understanding of the fundamental concepts that underpin modern computing systems.